

INTRODUÇÃO A BANCOS DE DADOS

Utilizando Python e SQLite

Duração:	120 minutos
Nível:	Iniciante
Tecnologias:	Python, SQLite, SQL
Pré-requisitos:	Conhecimentos básicos de Python

SUMÁRIO

1. O que é um Banco de Dados?
2. Conceito de Banco de Dados Relacional
3. O que é SQL? Introdução ao SQLite
4. Conceitos Fundamentais: Tabelas, Registros e Chaves
5. Tipos de Dados em SQL
6. Comandos SQL Essenciais
 - 6.1. CREATE DATABASE e USE DATABASE
 - 6.2. CREATE TABLE
 - 6.3. DROP TABLE
 - 6.4. INSERT INTO
 - 6.5. SELECT
7. Exercício Prático: Sistema de Escola
8. Referências e Próximos Passos

1. O QUE É UM BANCO DE DADOS?

Um **banco de dados** é um sistema organizado para armazenar, gerenciar e recuperar informações de forma eficiente. Pense nele como um armário digital onde guardamos dados de maneira estruturada.

Diferenças entre Arquivos Simples e Bancos de Dados

Característica	Arquivos (txt, csv)	Banco de Dados
Estrutura	Não estruturada ou semi-estruturada	Altamente estruturada
Consultas	Leitura sequencial	Consultas complexas e rápidas
Integridade	Sem validação automática	Regras de integridade
Concorrência	Difícil controlar acessos simultâneos	Gerencia múltiplos usuários
Segurança	Baseada em permissões de arquivo	Controle granular de acesso
Backup/Recovery	Manual	Automatizado

Exemplo prático: Imagine uma lista de contatos. Em um arquivo .txt, você teria apenas texto solto. Em um arquivo .csv, teria colunas separadas por vírgula. Em um banco de dados, você teria uma estrutura robusta com validações, relacionamentos e capacidade de fazer consultas complexas como 'encontre todos os contatos de São Paulo que nasceram após 1990'.

2. CONCEITO DE BANCO DE DADOS RELACIONAL

Um **Banco de Dados Relacional (BDR)** organiza as informações em **tabelas** que se relacionam entre si através de **chaves**. É o tipo mais comum de banco de dados utilizado atualmente.

Princípios Fundamentais

- 1. Dados organizados em tabelas:** Cada tabela representa uma entidade (ex: Clientes, Produtos, Pedidos).
- 2. Linhas e Colunas:** Cada linha é um registro único, cada coluna é um atributo.
- 3. Relacionamentos:** Tabelas se conectam através de chaves, evitando redundância.
- 4. Integridade referencial:** Garantia de que relacionamentos entre tabelas permanecem consistentes.

Exemplo Visual de Tabela

id_aluno	nome	idade	cidade
1	Ana Silva	20	São Paulo
2	Carlos Souza	22	Rio de Janeiro
3	Maria Santos	19	Salvador

Neste exemplo, cada linha representa um aluno, e cada coluna um atributo desse aluno.

3. O QUE É SQL? INTRODUÇÃO AO SQLITE

O que é SQL?

SQL (Structured Query Language) é a linguagem padrão para interagir com bancos de dados relacionais. Com SQL, você pode:

- Criar e modificar estruturas de dados (CREATE, ALTER, DROP)
- Inserir, atualizar e deletar dados (INSERT, UPDATE, DELETE)
- Consultar e recuperar informações (SELECT)
- Controlar permissões e segurança (GRANT, REVOKE)

O que é SQLite?

SQLite é um sistema de gerenciamento de banco de dados relacional que:

- **Não requer servidor:** O banco é apenas um arquivo no disco
- **É leve:** Ideal para aprendizado e aplicações pequenas/médias
- **Vem integrado ao Python:** Módulo sqlite3 nativo
- **É gratuito:** Domínio público, sem licenças

Conectando ao SQLite em Python

```
import sqlite3

# Conectar ao banco (cria se não existir)
conexao = sqlite3.connect('meu_banco.db')

# Criar cursor para executar comandos
cursor = conexao.cursor()

# Executar comandos SQL
cursor.execute("SELECT sqlite_version();")
versao = cursor.fetchone()
print(f"Versão do SQLite: {versao[0]}")

# Fechar conexão
conexao.close()
```

Importante: Sempre feche a conexão após usar o banco de dados para liberar recursos!

4. CONCEITOS FUNDAMENTAIS

Tabelas

Uma **tabela** é uma coleção de dados organizados em linhas e colunas. Representa uma entidade do mundo real (ex: Alunos, Produtos, Vendas).

Registros (Linhas)

Um **registro** é uma linha na tabela que representa uma instância específica da entidade. Por exemplo, um aluno específico na tabela de Alunos.

Colunas (Campos)

Uma **coluna** representa um atributo ou característica da entidade. Por exemplo: nome, idade, email.

Chave Primária (Primary Key - PK)

A **chave primária** é um campo (ou conjunto de campos) que identifica UNICAMENTE cada registro na tabela. Características:

- Deve ser única para cada registro
- Não pode ter valores nulos (NULL)
- Geralmente é um número inteiro auto-incrementável
- Exemplo: id_aluno, cpf, matricula

Chave Estrangeira (Foreign Key - FK)

A **chave estrangeira** é um campo que cria um relacionamento entre duas tabelas. Ela referencia a chave primária de outra tabela.

Exemplo de Relacionamento

Tabela: Alunos

id_aluno (PK)	nome	idade
1	Ana Silva	20
2	Carlos Souza	22

Tabela: Matriculas

id_matricula (PK)	id_aluno (FK)	id_turma (FK)	data_matricula
1	1	101	2024-01-15
2	2	102	2024-01-16

Note que **id_aluno** na tabela Matriculas é uma chave estrangeira que referencia a chave primária **id_aluno** da tabela Alunos.

5. TIPOS DE DADOS EM SQL

Cada coluna em uma tabela deve ter um tipo de dado definido. Isso garante a integridade e otimiza o armazenamento.

Tipo	Descrição	Exemplo
INT / INTEGER	Números inteiros	42, -10, 0
VARCHAR(n)	Texto de tamanho variável até n caracteres	VARCHAR(100) → "João Silva"
TEXT	Texto de tamanho ilimitado	Descrições longas, comentários
DATE	Data no formato YYYY-MM-DD	2024-01-30
DATETIME	Data e hora	2024-01-30 14:30:00
BOOLEAN	Verdadeiro ou Falso (0 ou 1)	TRUE, FALSE
REAL / FLOAT	Números com casas decimais	3.14, 10.5
BLOB	Dados binários (imagens, arquivos)	Arquivos binários

Exemplo de Definição de Tipos

```
CREATE TABLE produtos (  
  id_produto INTEGER PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  descricao TEXT,  
  preco REAL,  
  quantidade_estoque INT,  
  disponivel BOOLEAN,  
  data_cadastro DATE  
);
```

Dica: Escolha o tipo de dado mais apropriado para cada campo. Isso melhora o desempenho e previne erros de inserção.

6. COMANDOS SQL ESSENCIAIS

6.1. CREATE DATABASE e USE DATABASE

Nota sobre SQLite: Diferente de outros SGBDs (MySQL, PostgreSQL), o SQLite não usa os comandos CREATE DATABASE e USE DATABASE. Em vez disso, o banco de dados é criado automaticamente ao conectar.

```
import sqlite3

# Cria/conecta ao banco 'escola.db'
conexao = sqlite3.connect('escola.db')
print("Banco de dados 'escola.db' criado/conectado com sucesso!")

# Para outros SGBDs (MySQL, PostgreSQL), seria:
# CREATE DATABASE escola;
# USE escola;
```

6.2. CREATE TABLE

O comando **CREATE TABLE** cria uma nova tabela no banco de dados.

```
# Sintaxe básica
CREATE TABLE nome_tabela (
    coluna1 TIPO [restrições],
    coluna2 TIPO [restrições],
    ...
);

# Exemplo prático
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

cursor.execute('''
CREATE TABLE alunos (
    id_aluno INTEGER PRIMARY KEY AUTOINCREMENT,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    data_nascimento DATE,
    ativo BOOLEAN DEFAULT 1
)''')

conexao.commit()
print("Tabela 'alunos' criada com sucesso!")
conexao.close()
```

Restrições importantes:

- **PRIMARY KEY:** Define a chave primária
- **AUTOINCREMENT:** Incrementa automaticamente o valor
- **NOT NULL:** Campo obrigatório (não aceita valores nulos)
- **UNIQUE:** Valor deve ser único na tabela
- **DEFAULT:** Valor padrão se nenhum for fornecido

6.3. DROP TABLE

O comando **DROP TABLE** remove permanentemente uma tabela e todos os seus dados. **Use com extremo cuidado!**

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# CUIDADO: Isto apaga a tabela permanentemente!
cursor.execute('DROP TABLE IF EXISTS alunos')

conexao.commit()
print("Tabela 'alunos' removida.")
conexao.close()

# IF EXISTS evita erros se a tabela não existir
```

Atenção: DROP TABLE é irreversível! Todos os dados serão perdidos. Sempre faça backup antes de executar este comando em produção.

6.4. INSERT INTO

O comando **INSERT INTO** adiciona novos registros à tabela.

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Inserção especificando colunas
cursor.execute('''
INSERT INTO alunos (nome, email, data_nascimento, ativo)
VALUES (?, ?, ?, ?)
''', ('Ana Silva', 'ana@email.com', '2004-03-15', 1))

# Inserir múltiplos registros
alunos = [
    ('Carlos Souza', 'carlos@email.com', '2003-07-20', 1),
    ('Maria Santos', 'maria@email.com', '2005-11-10', 1),
    ('João Oliveira', 'joao@email.com', '2004-01-25', 1)
]

cursor.executemany('''
INSERT INTO alunos (nome, email, data_nascimento, ativo)
VALUES (?, ?, ?, ?)
''', alunos)

conexao.commit()
print(f"{cursor.rowcount} registros inseridos com sucesso!")
conexao.close()
```

Boas práticas: Use placeholders (?) para evitar SQL Injection. Nunca concatene valores diretamente na query!

6.5. SELECT - Consultando Dados

O comando **SELECT** recupera dados do banco de dados. É o comando mais utilizado em SQL.

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Selecionar todas as colunas de todos os registros
cursor.execute('SELECT * FROM alunos')
todos_alunos = cursor.fetchall()

print("Todos os alunos:")
for aluno in todos_alunos:
    print(aluno)

print("\n" + "="*50 + "\n")

# Selecionar colunas específicas
cursor.execute('SELECT nome, email FROM alunos')
nomes_emails = cursor.fetchall()

print("Nomes e e-mails:")
for nome, email in nomes_emails:
    print(f"Nome: {nome}, Email: {email}")

print("\n" + "="*50 + "\n")

# Selecionar com condição (WHERE)
cursor.execute('SELECT * FROM alunos WHERE ativo = 1')
alunos_ativos = cursor.fetchall()

print(f"Alunos ativos: {len(alunos_ativos)}")

# Selecionar apenas um registro
cursor.execute('SELECT * FROM alunos WHERE id_aluno = 1')
um_aluno = cursor.fetchone()
print(f"\nPrimeiro aluno: {um_aluno}")

conexao.close()
```

Métodos para recuperar dados:

- **fetchall()**: Retorna todos os resultados como lista
- **fetchone()**: Retorna apenas um registro
- **fetchmany(n)**: Retorna n registros

7. EXERCÍCIO PRÁTICO: SISTEMA DE ESCOLA

Agora vamos aplicar tudo o que aprendemos criando um sistema simples de gerenciamento escolar com duas tabelas relacionadas: **Alunos** e **Turmas**.

Objetivo do Exercício

1. Criar um banco de dados chamado 'escola.db'
2. Criar as tabelas Turmas e Alunos
3. Inserir dados nas duas tabelas
4. Realizar consultas para visualizar os dados
5. Consultar alunos de uma turma específica

Estrutura das Tabelas

Tabela: turmas

Campo	Tipo	Restrições
id_turma	INTEGER	PRIMARY KEY AUTOINCREMENT
nome_turma	VARCHAR(50)	NOT NULL
ano	INTEGER	NOT NULL
periodo	VARCHAR(20)	

Tabela: alunos

Campo	Tipo	Restrições
id_aluno	INTEGER	PRIMARY KEY AUTOINCREMENT
nome	VARCHAR(100)	NOT NULL
data_nascimento	DATE	
id_turma	INTEGER	FOREIGN KEY → turmas(id_turma)

8. REFERÊNCIAS E PRÓXIMOS PASSOS

Documentação Oficial

- SQLite: <https://www.sqlite.org/docs.html>
- Python sqlite3: <https://docs.python.org/3/library/sqlite3.html>
- SQL Tutorial: <https://www.w3schools.com/sql/>

Próximos Tópicos para Estudo

- Cláusulas WHERE, ORDER BY, LIMIT
- Funções de agregação (COUNT, SUM, AVG, MAX, MIN)
- JOINS avançados (LEFT, RIGHT, FULL OUTER)
- UPDATE e DELETE de registros
- Índices para otimização
- Transações (BEGIN, COMMIT, ROLLBACK)
- Normalização de bancos de dados
- ORMs (Object-Relational Mapping) como SQLAlchemy

Resumo da Aula

- ✓ Conceitos fundamentais de bancos de dados relacionais
- ✓ Diferenças entre arquivos e bancos estruturados
- ✓ Introdução ao SQL e SQLite
- ✓ Tabelas, registros, colunas e chaves (PK e FK)
- ✓ Tipos de dados principais (INT, VARCHAR, DATE, BOOLEAN)
- ✓ Comandos essenciais: CREATE TABLE, DROP TABLE, INSERT, SELECT
- ✓ Exercício prático com sistema de escola

Parabéns! Você concluiu a introdução a bancos de dados com Python. Continue praticando e explorando novos conceitos!